```
a();

print "never reached";

Output:
Caught in B...
Uncaught in A...
PHP Fatal error:  Uncaught Exception: Thrown in C... in...
```

The call chain is `a()` calls `b()`, which calls `c()`... three calls deep. `c()` throws an exception but has a `try` block, but `b()` does so the exception is caught there. Remember that I said a `catch` block could also `throw`? That's exactly what's going on here. The same exception object is then thrown from `b()`, and this time is not caught in `a()`. Instead, we have a `finally` which executes its code, then the exception is handled: the default handler spits out an `E_ERROR` as usual and kills the script.

Now, if you've followed any of these code examples at home, you'll know that I've been omitting some of the default handler's output: the *Stack trace*. The `E_ERROR` message is actually:

```
PHP Fatal error:  Uncaught Exception: Thrown in C... in /usr/src/
myapp/05.11_call_chain.php:22
Stack trace:
#0 /usr/src/myapp/call_chains.php(14): c()
#1 /usr/src/myapp/call_chains.php(6): b()
#2 /usr/src/myapp/call_chains.php(25): a()
#3 {main}
  thrown in ...
```

You'll see these on all default `E_ERROR` messages from the default exception handler. It's standard fare in exception-based languages to dump a stack trace; go take a look at Java sometime (if you think PHP traces are a bit verbose, you're in for a shock). Each line of the trace output is numbered, with the call that produced the error at the top as the zeroth line. We threw the exception in `c()`, with the message `'Thrown in C...'`, and that's what the error message contains. The stack trace that follows is like a map that shows us how we got to the error state.